

MANUAL TÉCNICO

Quibdó Seguro

Plataforma de reportes ciudadanos y gestión de recompensas

Versión	1.0
Fecha	Junio 2026
Autores	Equipo de Desarrollo
Institución	Fundación Universitaria Claretiana — Uniclaretiana
Tecnologías	Laravel · Node.js · MongoDB · Docker

Tabla de Contenidos

- 1. Descripción General del Sistema
- 2. Arquitectura del Sistema
- 3. Microservicios y Puertos
- 4. Variables de Entorno
- 5. Endpoints Principales
- 6. Base de Datos (MongoDB)
- 7. Docker Compose — Entorno de Desarrollo
- 8. Despliegue en Producción
- 9. Seguridad y Autenticación
- 10. Pruebas Recomendadas
- 11. Guía de Contribución

1. Descripción General del Sistema

Quibdó Seguro es una plataforma web orientada a la seguridad ciudadana en el municipio de Quibdó, Chocó, Colombia. Permite a ciudadanos reportar incidentes de seguridad, visualizarlos en un mapa interactivo y obtener recompensas por su participación activa. El sistema también incluye módulos para comercios aliados y administradores.

La aplicación está dividida en dos capas principales: una interfaz de usuario construida con Laravel (monolito UI) y un conjunto de microservicios independientes que manejan la lógica de negocio, comunicados a través de un API Gateway.

2. Arquitectura del Sistema

La arquitectura sigue el patrón de **microservicios con API Gateway**. El frontend (Laravel Blade) se comunica exclusivamente con el Gateway en el puerto 8000, quien enruta las peticiones al microservicio correspondiente.

Capa	Tecnología	Responsabilidad
Frontend / UI	Laravel 10 + Blade	Renderizado de vistas, sesiones de usuario
API Gateway	Node.js / Express	Enrutamiento, rate limiting, CORS
Auth Service	Node.js / Express	Registro, login, JWT, perfil
Incident Service	Node.js / Express	CRUD de incidentes, geolocalización
Rewards Service	Node.js / Express	Puntos, ofertas, canjes
Notification Service	Node.js / Express	Notificaciones push/email
Analytics Service	Node.js / Express	Estadísticas y métricas
Base de Datos	MongoDB 6	Almacenamiento persistente de todos los servicios

3. Microservicios y Puertos

Servicio	Puerto	Base URL
API Gateway	8000	http://localhost:8000
Auth Service	8001	http://localhost:8001
Incident Service	8002	http://localhost:8002
Rewards Service	8003	http://localhost:8003
Notification Service	8004	http://localhost:8004
Analytics Service	8005	http://localhost:8005
MongoDB	27017	mongodb://localhost:27017
Laravel UI	80 / 8080	http://localhost:8080

4. Variables de Entorno

Cada servicio lee su configuración de un archivo **.env**. A continuación se listan las variables críticas:

Variable	Descripción	Ejemplo
GATEWAY_URL	URL del API Gateway desde Laravel	http://gateway:8000
MONGODB_URI	Cadena de conexión a MongoDB	mongodb://mongo:27017/quibdo_seguro
DB_DATABASE	Nombre de la base de datos	quibdo_seguro
SESSION_DRIVER	Driver de sesiones Laravel	database
LOG_CHANNEL	Canal de logs	stack
JWT_SECRET	Clave secreta para tokens JWT	(generar con openssl rand -hex 32)
SANCTUM_STATEFUL_DOMAINS	Dominios permitidos Sanctum	localhost,127.0.0.1
APP_KEY	Clave de aplicación Laravel	(php artisan key:generate)
APP_URL	URL base de la aplicación	http://localhost:8080

5. Endpoints Principales

Todos los endpoints se exponen a través del API Gateway (puerto 8000). El Gateway autentica el token y reenvía la solicitud al microservicio correspondiente.

Método	Endpoint	Servicio	Descripción
POST	/auth/register	auth-service	Registro de nuevo usuario
POST	/auth/login	auth-service	Inicio de sesión, retorna JWT
GET	/auth/profile	auth-service	Perfil del usuario autenticado
POST	/auth/logout	auth-service	Cierre de sesión
POST	/incidentes	incident-service	Crear nuevo reporte de incidente
GET	/incidentes	incident-service	Listar todos los incidentes
GET	/incidentes/{id}	incident-service	Detalle de un incidente
PUT	/incidentes/{id}	incident-service	Actualizar incidente
DELETE	/incidentes/{id}	incident-service	Eliminar incidente
GET	/rewards/offers	rewards-service	Listar ofertas disponibles
POST	/rewards/redeem	rewards-service	Canjear recompensa
GET	/rewards/points	rewards-service	Consultar puntos del usuario
GET	/stats	analytics-service	Estadísticas generales
GET	/stats/map	analytics-service	Datos para el mapa de calor
POST	/notifications/send	notification-service	Enviar notificación

6. Base de Datos (MongoDB)

El sistema utiliza MongoDB 6 como motor de base de datos para todos los microservicios. Cada servicio tiene su propia colección dentro de la base de datos **quibdo_seguro**.

Colecciones principales

Colección	Servicio propietario	Campos principales
users	auth-service	_id, name, email, password_hash, role, points, created_at
incidentes	incident-service	_id, user_id, tipo, descripcion, ubicacion (GeoJSON), estado, created_at
rewards_offers	rewards-service	_id, titulo, descripcion, puntos_requeridos, comercio_id, activo
redemptions	rewards-service	_id, user_id, offer_id, fecha, estado
notifications	notification-service	_id, user_id, mensaje, leida, created_at
analytics_events	analytics-service	_id, tipo, payload, timestamp

Índices recomendados

```
// Índice geoespacial para incidentes (mapa)
db.incidentes.createIndex({ ubicacion: "2dsphere" })

// Índice para búsqueda por usuario
db.incidentes.createIndex({ user_id: 1 })
```

```
db.redemptions.createIndex({ user_id: 1 })
db.notifications.createIndex({ user_id: 1, leida: 1 })

// TTL para notificaciones antiguas (90 días)
db.notifications.createIndex({ created_at: 1 }, { expireAfterSeconds: 7776000 })
```

7. Docker Compose — Entorno de Desarrollo

El archivo **docker-compose.dev.yml** levanta todos los servicios en contenedores para el entorno de desarrollo local. Para iniciar el sistema:

```
docker compose -f docker-compose.dev.yml up --build
```

Contenido del archivo docker-compose.dev.yml:

```
version: '3.8'
services:
  mongo:
    image: mongo:6
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db

  gateway:
    build: ./microservices/gateway
    ports:
      - "8000:8000"
    environment:
      - MONGODB_URI=mongodb://mongo:27017/quibdo_seguro

  auth-service:
    build: ./microservices/auth-service
    ports:
      - "8001:8001"
    environment:
      - MONGODB_URI=mongodb://mongo:27017/quibdo_seguro

  incident-service:
    build: ./microservices/incident-service
    ports:
      - "8002:8002"
    environment:
      - MONGODB_URI=mongodb://mongo:27017/quibdo_seguro

  rewards-service:
    build: ./microservices/rewards-service
    ports:
      - "8003:8003"
    environment:
      - MONGODB_URI=mongodb://mongo:27017/quibdo_seguro

  notification-service:
    build: ./microservices/notification-service
    ports:
      - "8004:8004"
    environment:
      - MONGODB_URI=mongodb://mongo:27017/quibdo_seguro

  analytics-service:
    build: ./microservices/analytics-service
    ports:
      - "8005:8005"
```

```
environment:
  - MONGODB_URI=mongodb://mongo:27017/quibdo_seguro

laravel-ui:
  build: ./laravel-ui
  ports:
    - "8080:80"
  environment:
    - GATEWAY_URL=http://gateway:8000
  depends_on:
    - gateway

volumes:
  mongo-data:
```

8. Despliegue en Producción

Para desplegar Quibdó Seguro en un entorno de producción se recomiendan las siguientes opciones:

8.1 Render / Railway (PaaS gratuito)

Ambas plataformas permiten desplegar servicios Node.js y PHP de forma gratuita con límites razonables:

- Crear una cuenta en render.com o railway.app
- Conectar el repositorio de GitHub/GitLab
- Configurar cada microservicio como un servicio independiente
- Agregar un servicio MongoDB (o usar MongoDB Atlas — plan gratuito M0)
- Establecer las variables de entorno en el panel de la plataforma
- Configurar la variable `GATEWAY_URL` apuntando a la URL de producción del gateway

8.2 MongoDB Atlas (base de datos gratuita)

MongoDB Atlas ofrece un clúster M0 gratuito con 512 MB de almacenamiento, suficiente para desarrollo y demos:

```
# String de conexión Atlas (reemplazar usuario/password/cluster)
MONGODB_URI=mongodb+srv://usuario:password@cluster0.xxxxx.mongodb.net/quibdo_seguro?retryWrites=true
```

8.3 Cloudflare Tunnel (exposición local temporal)

Para demos rápidas sin despliegue completo, Cloudflare Tunnel permite exponer localhost a internet:

```
# Instalar cloudflared y ejecutar
cloudflared tunnel --url http://localhost:8080
# Genera una URL pública temporal como: https://random-name.trycloudflare.com
```


9. Seguridad y Autenticación

El sistema implementa autenticación basada en **JWT (JSON Web Tokens)** gestionado por el Auth Service. El API Gateway valida el token en cada petición antes de reenviarla al microservicio destino.

Flujo de autenticación

Paso	Actor	Acción
1	Cliente (Laravel)	POST /auth/login con email y password
2	Gateway	Reenvía al auth-service
3	Auth Service	Valida credenciales, genera JWT con 24h de expiración
4	Gateway → Cliente	Retorna JWT
5	Cliente	Incluye JWT en header: Authorization: Bearer {token}
6	Gateway	Valida JWT en cada petición protegida
7	Microservicio	Recibe petición ya autenticada

Consideraciones de seguridad adicionales

- Usar HTTPS en producción (Let's Encrypt + Nginx o configuración de la plataforma PaaS)
 - Configurar CORS correctamente en el Gateway para restringir dominios permitidos
 - Almacenar contraseñas con bcrypt (salt factor ≥ 12)
 - Implementar rate limiting en el Gateway para prevenir ataques de fuerza bruta
 - Rotar el JWT_SECRET periódicamente e invalidar sesiones activas
 - Validar y sanitizar todos los inputs antes de insertarlos en MongoDB
 - No exponer el puerto de MongoDB (27017) directamente a internet
-

10. Pruebas Recomendadas

Pruebas unitarias

Para el backend Laravel, usar **PHPUnit** (ya incluido en Laravel):

```
php artisan test
```

Para los microservicios Node.js, usar **Jest**:

```
npm test # en cada directorio de microservicio
```

Pruebas E2E

Usar **Laravel Dusk** o **Cypress** para pruebas end-to-end del flujo completo:

```
# Instalar Cypress
npm install --save-dev cypress
```

```
# Ejecutar pruebas E2E
npx cypress open
```

Pruebas de carga

Para verificar el comportamiento bajo carga, usar **k6**:

```
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
  vus: 50,
  duration: '30s',
};

export default function () {
  http.get('http://localhost:8000/incidentes');
  sleep(1);
}

k6 run test_carga.js
```

11. Guía de Contribución

Para contribuir al proyecto, seguir el flujo estándar de Git Flow:

- Clonar el repositorio: `git clone https://github.com/org/quibdo-seguro.git`
- Crear una rama a partir de develop: `git checkout -b feature/nombre-feature`
- Desarrollar la funcionalidad con sus respectivas pruebas
- Ejecutar las pruebas localmente antes del push
- Abrir un Pull Request hacia la rama develop con descripción clara
- Esperar revisión de código (mínimo 1 aprobación requerida)
- Merge a develop → staging para pruebas → main para producción

Estilo de código: seguir las convenciones PSR-12 para PHP y las del proyecto para JavaScript (ESLint configurado). Todos los commits deben seguir el formato Conventional Commits: *feat.*, *fix.*, *docs.*, *chore.*, etc.